CM

GROUP 04 • MINI PROJECT REPORT

Campus Complaint and Maintenance Portal

A Flask and SQLite web application for registering, categorising and tracking campus maintenance complaints

Project detail	Information
Group	04
Member 1	Kush — 2026UCD3136
Member 2	Kinshuk — 2026UCD2173
Member 3	Akshat — 2026UCD3123
Institution	Netaji Subhas University of Technology (NSUT)
Project title	Campus Complaint and Maintenance Portal
Technologies	Python, Flask, SQLite, HTML, CSS, JavaScript
Academic year	2026

"Report it. Track it. Resolve it."

Project overview | architecture | database | API routes | testing plan | complete source code

Contents

1 Project Overview 3

2 Technologies and System Architecture 4

3 Database Design 5

4 Flask API Routes and HTTP Methods 6

5 Frontend Implementation and User Interface 8

6 Validation, Error Handling and Security 10

7 Testing Plan 11

8 Limitations, Future Scope and Conclusion 12

9 Installation, Running Instructions and Guideline Checklist 13

A Appendix A — Application Screenshots 15

B Appendix B — Filled Mini-Project Worksheet 16

C Appendix C — Complete Project Source Code 19

About this report

This report documents the Campus Complaint and Maintenance Portal developed by Group 04. Every feature, route and database field described here is taken from the project's source code, which is reproduced in full in Appendix C. Screenshots of the running application are collected in Appendix A, and the test plan in Section 7 lists each test case with its expected result and space to record the observed outcome.

1. Project Overview

The Campus Complaint and Maintenance Portal is a Flask-based web application for registering and tracking campus maintenance complaints. From a single browser page, a student or other reporter can lodge a complaint, browse and filter existing complaint records, update a complaint's status and resolution notes, delete a record, and see summary counts of complaints by status. All complaint data is stored persistently in a SQLite database.

Problem statement

Campus facility issues — electrical faults, plumbing problems, internet outages, damaged furniture and cleanliness concerns — need a simple way to be reported and followed up. When such reports are handled informally, the details of each issue are scattered and it becomes difficult to see which issues are still pending, which are being addressed and which have been resolved. This project provides a centralised web interface in which maintenance complaints are recorded in a consistent structure and their progress can be monitored at a glance.

Objectives

1. Provide a structured form for students or other reporters to submit campus maintenance complaints.
2. Categorise and display complaint records, with filtering by category and by status.
3. Allow a complaint's status and resolution notes to be updated, while dashboard totals show overall complaint progress.

Key features implemented

Feature	Description
Complaint submission	Form for name / roll no., issue summary, category, location and description — all five required.
Categories	Electrical, Plumbing, IT & Internet, Furniture / Carpentry, Cleanliness and Other.
Complaint list	Cards with title, status, location, category, reporter, description and any resolution note.
Filtering	Drop-downs filter by status and by category, sent to the backend as query parameters.
Status tracking	Pending In Progress Resolved colour-coded tags; new complaints start as Pending.
Resolution notes	An update dialog changes the status and records the action taken.
Deletion	A Delete button on each card, with a browser confirmation prompt.
Dashboard	Total, Pending, In Progress and Resolved counts, fetched from the backend.
Persistent storage	Records stored in the `complaints` table of the SQLite file `complaints.db`.

Scope and boundaries

The application implements a focused complaint-record workflow for a local demonstration. The following are outside the current scope and are **not** claimed as implemented: user login or role-based access; assigning complaints to technicians or departments; email, SMS or push notifications; photo or file attachments; maps or location tracking; analytics beyond the four dashboard counts; status-change history or audit logs; and deployment to a public server.

2. Technologies and System Architecture

Technology	Role in this project
Python	Backend application logic, validation and database operations (`app.py`, `database.py`).
Flask 3.0.0	Web framework: serves the HTML page and defines the API routes and HTTP methods.
SQLite	Persistent storage of complaint records through Python's built-in `sqlite3` module.
HTML	Structure of the single-page interface (`templates/index.html`).
CSS	Layout, typography, colours, cards, status tags and the responsive dashboard (`static/css/style.css`).
JavaScript	Form handling, filters, the update dialog and asynchronous `fetch()` requests (`static/js/app.js`).
JSON	Data format for requests and responses between the browser and the Flask API.

How the application works

Flask serves the page with `render_template('index.html')`; the page loads its stylesheet from `static/css/style.css` and its script from `static/js/app.js`. JavaScript then uses `fetch()` to send HTTP requests to the Flask API routes. Each route validates its input, reads from or writes to SQLite through `database.py`, and returns a JSON response built with `jsonify()`. The script uses the response to redraw the complaint cards and dashboard counts without reloading the page.

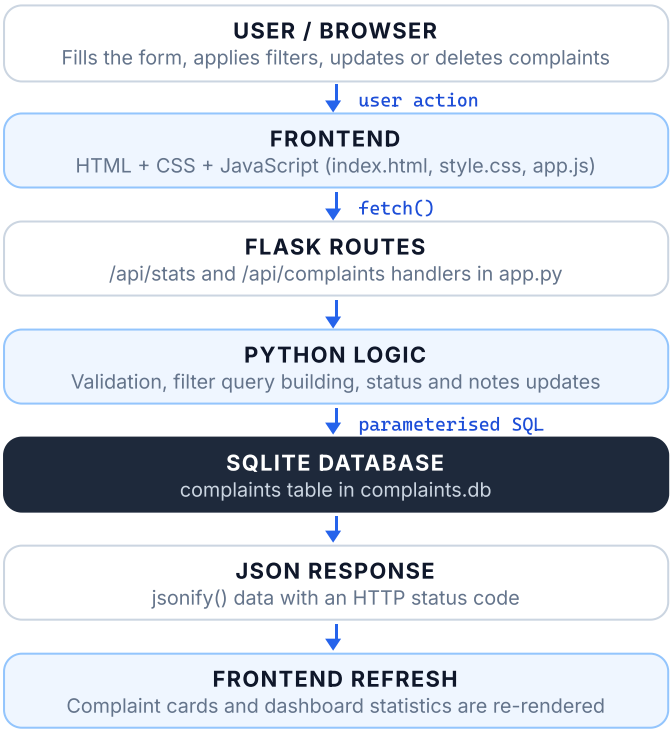


Figure 1. Request–response flow between the browser, Flask and SQLite.

Main user workflow

1. The user opens the portal in a browser.
2. The frontend requests the dashboard statistics and the complaint records.
3. The user fills in the complaint form and submits it.
4. JavaScript sends the complaint data as JSON to `POST /api/complaints`.
5. Flask validates the fields and stores the record in SQLite.
6. The frontend refreshes the complaint list and dashboard counts.
7. The user may filter records by category or status.
8. The user may update a complaint's status and resolution notes, or delete a record.
9. After each successful change the frontend refreshes the list and statistics again.

3. Database Design

The project uses SQLite through Python's built-in `sqlite3` module. `database.py` defines two functions: `get_db_connection()`, which opens `complaints.db` and sets `row_factory = sqlite3.Row` so that rows can be converted to dictionaries; and `init_db()`, which creates the `complaints` table if it does not already exist. `app.py` calls `init_db()` when the application starts, so the table is always available and existing data is preserved across restarts.

Table: complaints

Field	Type / constraint	Purpose
`id`	`INTEGER PRIMARY KEY AUTOINCREMENT`	Unique complaint identifier
`title`	`TEXT NOT NULL`	Short issue summary
`category`	`TEXT NOT NULL`	Complaint category
`location`	`TEXT NOT NULL`	Campus location / room
`description`	`TEXT NOT NULL`	Detailed issue description
`reporter_name`	`TEXT NOT NULL`	Name / roll number of the reporter
`status`	`TEXT DEFAULT 'Pending'`	Current workflow status
`resolution_notes`	`TEXT DEFAULT ''`	Notes about the action taken
`created_at`	`TIMESTAMP DEFAULT CURRENT_TIMESTAMP`	Record creation timestamp

How the database is used

- **Inserting:** the create route inserts only the five form fields. SQLite fills in `id`, the default status `'Pending'`, an empty `resolution_notes` value and the `created_at` timestamp.
- **Reading:** the list route selects all columns, optionally filtered by status and/or category, ordered by `id DESC` so the newest record comes first. Each `sqlite3.Row` is converted with `dict(row)` before being returned with `jsonify()`.
- **Counting:** the statistics route runs four `COUNT(*)` queries — one for all complaints and one for each status.
- **Updating and deleting:** both routes first check that the record exists, then run an `UPDATE` or `DELETE` on that `id` and commit the change.
- **Parameterised queries:** every value supplied by the user is passed through `?` placeholders rather than being joined into the SQL string.
- **Connections:** each request opens its own connection and closes it after the query completes.

> **Note on `created_at`:** the timestamp is stored and returned by the API, but the current complaint cards do not display it.

> **Supplied database:** the `complaints.db` file included with the project contains the `complaints` table schema and no complaint rows. This report therefore does not present any complaint counts as real usage data.

4. Flask API Routes and HTTP Methods

The application has one page-rendering route and six API routes. The page route returns HTML; every `/api` route returns JSON through `jsonify()` together with an explicit HTTP status code.

Endpoint	Method	Purpose	Success
`/`	`GET`	Renders `templates/index.html`.	200
`/api/stats`	`GET`	Returns total, pending, in-progress and resolved counts.	200
`/api/complaints`	`GET`	Lists complaints, newest first; optional `status` and `category` filters.	200
`/api/complaints`	`POST`	Validates the five fields and creates a complaint record.	201
`/api/complaints/<id>`	`GET`	Returns one complaint, or 404 if it does not exist.	200
`/api/complaints/<id>`	`PATCH` `PUT`	Updates the status and/or resolution notes.	200
`/api/complaints/<id>`	`DELETE`	Deletes an existing complaint record.	200

`<id>` is declared in `app.py` as `<int:complaint_id>`, so only whole-number IDs match these routes.

Why each HTTP method is used

- **GET** retrieves data without changing it: the page itself, the dashboard counts, the complaint list and a single complaint.
- **POST** creates a new complaint from the submitted form data.
- **PATCH / PUT** modify an existing complaint. The frontend sends `PATCH`; the same handler also accepts `PUT`. Both behave as a partial update — a field that is omitted keeps its current value.
- **DELETE** removes an existing complaint record.

Because the portal supports the full create–read–update–delete cycle, it uses all four method types in the college guidelines, adapted from the worksheet's generic `/api/items` example.

Example request and response formats

The field names below match those sent by `static/js/app.js`. The sample values are taken from the placeholder text in the form and update dialog and are shown for illustration only.

```
// POST /api/complaints
{
  "reporter_name": "Kush (2026UCD3136)",
  "title": "AC leaking water in Lab 3",
  "category": "Electrical",
  "location": "Block B, 2nd Floor, Room 204",
  "description": "<details of the issue>"
}
// 201 Created
{ "success": true,
  "message": "Complaint logged successfully",
  "id": <new record id> }
```

```
// PATCH /api/complaints/<id>
{
  "status": "Resolved",
  "resolution_notes": "Technician replaced
capacitor on Nov 3rd."
}
// 200 OK
{ "success": true,
  "message": "Complaint updated successfully" }

// GET /api/stats → 200 OK
{ "total": n, "pending": n,
  "in_progress": n, "resolved": n }
```

List and single-record responses share one shape: `{"success": true, "data": …}`, where `data` is an array of complaint objects for the list route and a single object for the `/<id>` route. Error responses use `{"success": false, "error": "<message>"}`.

5. Frontend Implementation and User Interface

The interface is a single HTML page. Its content is drawn and refreshed by `static/js/app.js`, which talks to the Flask API with `fetch()` and `async/await`.

Page layout

- **Navigation bar** — dark slate bar with the heading "Campus Maintenance & Complaint Portal" and the label "Group 4: Kush | Kinshuk | Akshat".
- **Statistics row** — four cards (Total Complaints, Pending, In Progress, Resolved) in a responsive grid.
- **Dashboard grid** — the "Lodge a Complaint" form on the left (380 px wide) and the "Active Campus Tickets" list on the right. Below 900 px the grid collapses to a single column.
- **Complaint cards** — each card shows the title, a colour-coded status tag, location, category, reporter and description, an optional resolution note, and Update Status and Delete buttons. The list scrolls once it is taller than 650 px.
- **Update dialog** — a modal overlay with a status drop-down, a resolution-notes text area and a Save Update button.

JavaScript functions

Function	Triggered by	API request	Effect
`fetchStats()`	Page load; after each change	`GET /api/stats`	Fills the four counters.
`fetchComplaints()`	Page load; filters; after each change	`GET /api/complaints`	Rebuilds the cards ("No complaints found." if empty).
`handleCreateComplaint()`	Form submit	`POST /api/complaints`	Resets the form, refreshes; alerts on error.
`openUpdateModal() / closeModal()`	Update Status / ×	—	Pre-fills and shows, or hides, the dialog.
`saveStatusUpdate()`	Save Update	`PATCH /api/complaints/<id>`	Closes the dialog and refreshes.
`deleteComplaint()`	Delete	`DELETE /api/complaints/<id>`	Asks for confirmation, then refreshes.
`escapeHtml()`	Before inserting user text	—	Escapes `&` `<` `>` `"` `'` as HTML entities.

Visual design

An off-white page, a dark slate navigation bar, white cards with subtle borders and shadows, a blue primary action colour and the Inter typeface; status colours are used consistently on counters and tags. Screenshots are in Appendix A.

Pending	In Progress	Resolved	Primary	Nav bar	Delete
#f59e0b	#3b82f6	#10b981	#2563eb	#1e293b	#ef4444

Implementation highlights

- **Dashboard always in step with the data.** After every successful create, update or delete, the script calls both `fetchComplaints()` and `fetchStats()`, so the cards and the four counts are always re-read from the database.
- **Flexible but safe filtering.** The list query starts from `WHERE 1=1` and appends `AND status = ?` and/or `AND category = ?` only for the filters supplied, keeping every value parameterised.
- **Independent partial updates.** The update handler keeps the existing status or notes when either is omitted, so each can be changed on its own.

6. Validation, Error Handling and Security

Input validation

- **In the browser:** all five complaint-form fields carry the HTML `required` attribute, so the browser will not submit an incomplete form.
- **Creating a complaint:** the POST route returns **400** if the JSON body is empty. It then trims whitespace from the title, category, location, description and reporter name and returns **422** ("All fields are required") if any of them is empty.
- **Updating a complaint:** the PATCH/PUT route returns **400** if the body is empty and **404** if the complaint does not exist.
- **Reading or deleting one complaint:** both routes return **404** ("Complaint not found") for an ID that does not exist. The `<int:complaint_id>` converter means only whole-number IDs match these routes.

HTTP status codes used

Status	Meaning in this application
200 OK	Statistics, list, single record, update or delete completed successfully.
201 Created	A new complaint was stored; the response includes its new ID.
400 Bad Request	The create or update request body was empty.
404 Not Found	No complaint exists with the requested ID (single read, update or delete).
422 Unprocessable	One or more of the five required complaint fields was blank.

Security measures in the code

- **SQL injection protection:** all filter values, form values and IDs are passed to SQLite through `?` placeholders.
- **HTML injection protection:** before user-entered text is placed into a complaint card, `escapeHtml()` converts `&`, `<`, `>`, `"` and `'` into HTML entities.
- **Consistent error format:** API errors are returned as JSON with `"success": false` and a readable `"error"` message, which the frontend shows to the user in an alert.

Validation boundaries

To describe the implementation precisely, the following checks are **not** performed by the current server code:

- The update route does not check that the new status is one of Pending, In Progress or Resolved; the drop-down in the interface limits the choices instead.
- The create route checks that fields are present and non-empty, but not their data type, maximum length, or that the category is one of the six listed categories.
- The list route does not check filter values against an allowed list; an unrecognised value simply returns no records.
- The application runs with Flask's development server in debug mode (`app.run(debug=True, port=5000)`), which is suitable for a local demonstration only.

7. Testing Plan

The table below is the test plan for the main features. Each case lists how to perform it and the result expected from the source code. The **Actual result / status** column is filled in after the test has been run on the local application.

> **Suggested tests to perform.** No automated test scripts or recorded test results were part of the project files, so no results are claimed here. Record what you observe in the last column (for example "Pass — card appeared" or "Fail — ..."), and keep terminal output or browser captures as evidence if the faculty asks for it.

#	Test case	How to perform	Expected result	Actual result / status
1	**Open home page**	Run `python app.py`; open http://127.0.0.1:5000	Page loads; counters and list load (empty list shows "No complaints found.")	
2	**Submit valid complaint**	Fill all five fields; click Submit Ticket	201; form resets; Total and Pending each increase by 1	
3	**Submit with a blank field**	Leave one field empty (browser), or POST with `"title": ""`	Browser blocks the form; API returns 422 "All fields are required"	
4	**New record in list**	Check the list after test 2	New card at the top with title, Pending tag, location, category, reporter and description	
5	**Filter by status**	Select Pending, In Progress and Resolved in turn	Only complaints with the selected status are shown	
6	**Filter by category**	Select each of the six categories in turn	Only complaints in the selected category are shown	
7	**Get one complaint**	Open `/api/complaints/<id>` for an existing ID	200 with `"success": true` and that record in `data`	
8	**Pending → In Progress**	Update Status → In Progress → Save Update	Tag turns blue; Pending −1 and In Progress +1	
9	**Add resolution notes**	Type notes in the dialog and save	Card shows "Resolution Note: ..."	
10	**Mark as Resolved**	Change the status to Resolved and save	Tag turns green; Resolved +1	
11	**Delete complaint**	Click Delete and confirm	200; the card disappears	
12	**Dashboard totals**	Compare the counters after tests 2, 8, 10 and 11	Counts always match the records shown with no filter applied	
13	**Non-existent ID**	Open `/api/complaints/99999`	404 with `"error": "Complaint not found"`	
14	**Empty JSON body**	POST `{}` to `/api/complaints`	400 with `"error": "Request body must be JSON"`	

Running the API-level tests

Tests 7 and 13 can be run by opening the URL directly in the browser. Tests 3 (API part) and 14 need a request with a JSON body, which can be sent from the browser's developer console while the portal is open, for example:

```
fetch('/api/complaints', {method: 'POST', headers: {'Content-Type': 'application/json'},
body: '{}'}).then(r => r.json()).then(console.log)
```

8. Limitations, Future Scope and Conclusion

Current limitations

- No login, authentication or role-based permissions; anyone using the page can update or delete complaints.
- Complaints are not assigned to a technician or maintenance department.
- No notification system (email, SMS or in-app).
- No image or file upload for complaints.
- No status-change history or audit log — only the current status and the latest resolution note are stored.
- Server-side validation checks that fields are present, but not their allowed values (see Section 6).
- The application runs on Flask's development server for local use and is not deployed.
- The supplied database contains no complaint records, so no real usage data has been collected.

Possible future enhancements

The following are proposals for future work and are **not** part of the current implementation:

- Student and maintenance/admin login with role-based permissions.
- Assignment of complaints to a maintenance department or staff member.
- Email or in-app notifications when a complaint's status changes.
- Photo uploads for maintenance issues, and priority or severity levels.
- Search by keyword and date range, and display of the creation date on each card.
- A status-change history with timestamps, advanced analytics and downloadable reports.
- Accessibility and usability improvements, and deployment to a secure campus server.

Conclusion

The Campus Complaint and Maintenance Portal combines a Python and Flask backend, a SQLite database and an HTML, CSS and JavaScript frontend to give campus maintenance complaints a single, structured home. Users can lodge complaints, view and filter them, move them through Pending, In Progress and Resolved with resolution notes, delete records, and follow overall progress through live dashboard counts. The project meets the mini-project requirements for a Flask backend, RESTful routes with GET, POST, PATCH/PUT and DELETE, JSON responses, input validation and a frontend integrated through API requests.

Learning outcomes

The project demonstrates Flask routing with URL converters and multiple HTTP methods; SQLite table design and create, read, update and delete operations with parameterised queries; JSON request and response handling with `jsonify()` and HTTP status codes; server-side input validation; asynchronous JavaScript with `fetch()` and `async`/`await`; dynamic DOM updates with HTML escaping; and a responsive layout using CSS Grid.

9. Installation, Running Instructions and Guideline Checklist

Install and start

Requirements: Python 3.8 or later. The only third-party dependency, listed in `requirements.txt`, is `Flask==3.0.0`; SQLite support is built into Python. Open a terminal in the `project/` folder (the one containing `app.py`), install the dependency, then start the server:

```
python -m pip install -r requirements.txt
python app.py
```

Open **http://127.0.0.1:5000** in a browser while the terminal stays open. Stop the server with **Ctrl + C**. If Windows does not recognise `python`, try `py`.

> **Run from the project folder.** `database.py` opens `complaints.db` by a relative path, so the database is created in whichever folder the terminal is in. Starting the app from another folder creates a new, empty database there. Do not open `index.html` directly — it uses Flask template tags and must be served by Flask.

Project files

File / folder	Purpose
`app.py`	Flask application, page route and the six API routes.
`database.py`	SQLite connection helper and `init_db()` table creation.
`complaints.db`	SQLite database file containing the `complaints` table.
`requirements.txt`	Dependency declaration: `Flask==3.0.0`.
`templates/index.html`	Page structure: navigation bar, statistics, form, list and update dialog.
`static/css/style.css`	Visual design, status colours and responsive layout.
`static/js/app.js`	Browser interactions and `fetch()` API requests.

Guideline checklist

College requirement	Where it is addressed
Python backend using Flask	`app.py`, `database.py`; Sections 2–4; Appendix C.
Routes with GET, POST, PUT/PATCH, DELETE	Section 4 — all four method types are used.
API responses in JSON format	Section 4 — every `/api` route uses `jsonify()`.
Basic input validation and error handling	Section 6.
Frontend connected through API requests	Section 5; `static/js/app.js`.
Database such as SQLite where required	Section 3.
Testing of the main features	Section 7 — test plan with expected and observed results.
Source code with basic run instructions	This section; Appendix C.

Screenshots and working demonstration	Appendix A; local demonstration at http://127.0.0.1:5000.

Appendix A. Application Screenshots

Screens captured from the local Flask application at http://127.0.0.1:5000 after adding a few test complaints.

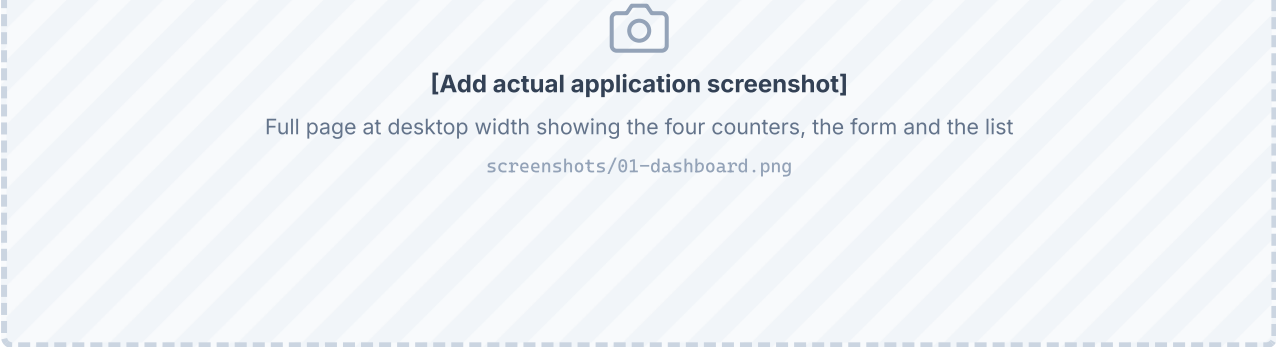


Figure A1. Full dashboard — statistics, complaint form and complaint list

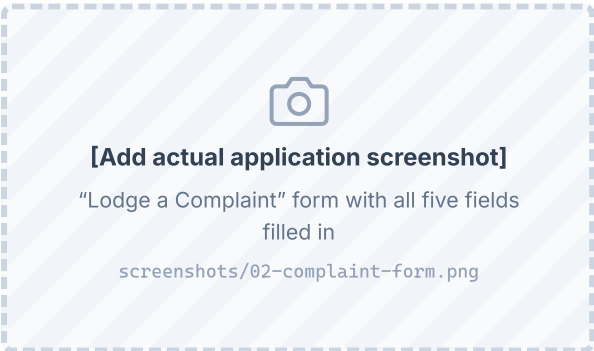


Figure A2. Complaint submission form

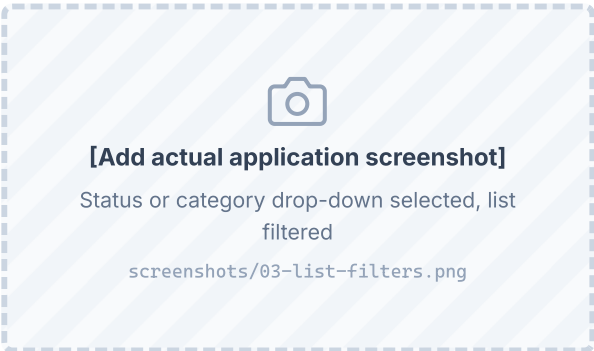


Figure A3. Complaint list with a filter applied

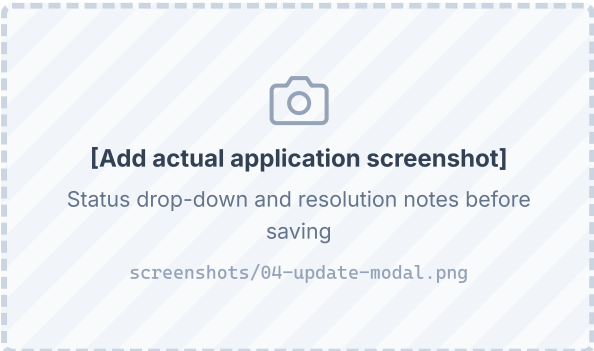


Figure A4. Update-status dialog

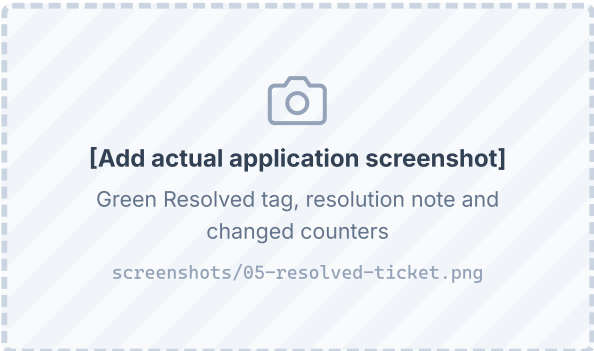


Figure A5. Resolved complaint and updated counts

Appendix B. Filled Mini-Project Worksheet

1. Project details

Worksheet field	Filled project details
Student names / roll numbers	Kush (2026UCD3136), Kinshuk (2026UCD2173), Akshat (2026UCD3123)
Project title	Campus Complaint and Maintenance Portal
Problem statement	Campus issues such as electrical faults, plumbing, internet, damaged furniture and cleanliness need a simple way to be reported and followed up; informal reporting makes it hard to see whether an issue is pending, being addressed or resolved.
Objectives	(1) A structured form to submit complaints. (2) Categorise and display complaints, filterable by category and status. (3) Update status and resolution notes, with dashboard totals showing progress.
Key features	Five-field complaint form; six categories; filterable complaint cards; status and notes dialog; delete with confirmation; four dashboard counts; SQLite storage.

2. Development roadmap

Phase	What was developed
Phase 1 — Backend	Flask application and SQLite database (`init_db()`); create, list with filters, single lookup, status/notes update, delete and statistics, exposed as JSON routes with GET, POST, PATCH/PUT and DELETE, plus validation and error responses.
Phase 2 — Frontend	Single-page HTML/CSS/JavaScript dashboard — statistics cards, complaint form, filterable cards and update dialog — connected to the APIs with `fetch()`; run instructions for a local demonstration.

3. Basic requirements

Requirement	Filled response
Backend and API	Python and Flask; GET to read the page, statistics and complaints; POST to create; PATCH/PUT to update status and notes; DELETE to remove a complaint.
JSON and validation	All `/api` routes return JSON. Empty bodies return 400, blank required fields return 422 and unknown IDs return 404.
Frontend	Responsive HTML/CSS dashboard; JavaScript `fetch()` requests update the page without reloading.
Database	SQLite table `complaints` in `complaints.db`, accessed with Python's `sqlite3` module.
Testing	Test plan of 14 cases in Section 7, covering every route and user action.

4. API planning

Endpoint	Method	Purpose
`/`	GET	Display the main page
`/api/stats`	GET	Dashboard counts

`/api/complaints`	GET	Retrieve complaints (optional filters)
`/api/complaints`	POST	Add a new complaint
`/api/complaints/<id>`	GET	Retrieve one complaint
`/api/complaints/<id>`	PATCH / PUT	Update status and resolution notes
`/api/complaints/<id>`	DELETE	Delete a complaint

5. Final submission checklist

Checklist item	Status / evidence
Backend implemented using Python and Flask	Implemented — `app.py`, `database.py` (Appendix C)
API routes created and tested	Routes created (Section 4); test plan in Section 7
Frontend integrated with the backend	Implemented — `static/js/app.js` (Section 5)
Main features working as expected	To be confirmed by running the Section 7 tests
Source code submitted	Appendix C and the project folder
Basic instructions for running the application	Section 9
Screenshots and working demonstration ready	Appendix A; local demonstration

6. Project discussion

Discussion point	Response
Problem and objectives	Give campus maintenance complaints one structured place to be reported, categorised and tracked from Pending to Resolved.
Technologies and purpose	Python and Flask handle routes and logic; SQLite stores complaints; HTML and CSS build the dashboard; JavaScript calls the APIs; JSON carries the data.
Routes and HTTP methods	GET reads the page, statistics and complaints; POST creates; PATCH/PUT update status and notes; DELETE removes a complaint.
Frontend–backend communication	JavaScript `fetch()` sends JSON requests to Flask routes; Flask returns JSON, and the page re-renders the cards and counts.
Testing and challenges	14-case test plan (Section 7). Counts stay consistent by re-fetching list and statistics after every change; filters are added to the SQL only when selected, keeping values parameterised.
Possible improvements	Logins and roles, staff assignment, notifications, photo uploads, priority levels, search, status history and campus-server deployment.

7. Group contributions

Member	Contribution
Kush (2026UCD3136)	[To be confirmed by the group]
Kinshuk (2026UCD2173)	[To be confirmed by the group]
Akshat (2026UCD3123)	[To be confirmed by the group]

> **Note:** individual contributions were not supplied with the project files. All members should agree on this table and be ready to explain the complete application.

Appendix C. Complete Project Source Code

The application files are reproduced below exactly as supplied. Line numbers are added for reference; long lines wrap to fit the page. The project folder remains the runnable copy.

app.py — Flask application and API routes · 166 lines

```
1  from flask import Flask, request, jsonify, render_template
2  from database import get_db_connection, init_db
3
4  app = Flask(__name__)
5
6  # Initialize database table on application start
7  init_db()
8
9  @app.route('/')
10 def home():
11     """Serves the single-page application frontend."""
12     return render_template('index.html')
13
14 # ----------------- RESTful API ROUTES ----------------- #
15
16 @app.route('/api/stats', methods=['GET'])
17 def get_stats():
18     """Returns counts of total, pending, in-progress, and resolved complaints."""
19     conn = get_db_connection()
20     cursor = conn.cursor()
21
22     cursor.execute('SELECT COUNT(*) FROM complaints')
23     total = cursor.fetchone()[0]
24
25     cursor.execute("SELECT COUNT(*) FROM complaints WHERE status = 'Pending'")
26     pending = cursor.fetchone()[0]
27
28     cursor.execute("SELECT COUNT(*) FROM complaints WHERE status = 'In Progress'")
29     in_progress = cursor.fetchone()[0]
30
31     cursor.execute("SELECT COUNT(*) FROM complaints WHERE status = 'Resolved'")
32     resolved = cursor.fetchone()[0]
33
34     conn.close()
35     return jsonify({
36         "total": total,
37         "pending": pending,
38         "in_progress": in_progress,
39         "resolved": resolved
40     }), 200
41
42 @app.route('/api/complaints', methods=['GET'])
43 def get_complaints():
44     """Retrieve all complaints with optional filtering."""
45     status_filter = request.args.get('status')
46     category_filter = request.args.get('category')
47
48     query = "SELECT * FROM complaints WHERE 1=1"
49     params = []
50
51     if status_filter:
52         query += " AND status = ?"
53         params.append(status_filter)
54     if category_filter:
55         query += " AND category = ?"
56         params.append(category_filter)
57
58     query += " ORDER BY id DESC"
59
60     conn = get_db_connection()
61     cursor = conn.cursor()
```

```
    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()

    complaints = [dict(row) for row in rows]
    return jsonify({"success": True, "data": complaints}), 200

@app.route('/api/complaints/<int:complaint_id>', methods=['GET'])
def get_complaint(complaint_id):
    """Retrieve details of a single complaint."""
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM complaints WHERE id = ?", (complaint_id,))
    row = cursor.fetchone()
    conn.close()

    if row is None:
        return jsonify({"success": False, "error": "Complaint not found"}), 404

    return jsonify({"success": True, "data": dict(row)}), 200

@app.route('/api/complaints', methods=['POST'])
def create_complaint():
    """Register a new complaint."""
    data = request.get_json()

    if not data:
        return jsonify({"success": False, "error": "Request body must be JSON"}), 400

    title = data.get('title', '').strip()
    category = data.get('category', '').strip()
    location = data.get('location', '').strip()
    description = data.get('description', '').strip()
    reporter_name = data.get('reporter_name', '').strip()

    # Input Validation
    if not all([title, category, location, description, reporter_name]):
        return jsonify({"success": False, "error": "All fields are required"}), 422

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO complaints (title, category, location, description, reporter_name)
        VALUES (?, ?, ?, ?, ?)
    ''', (title, category, location, description, reporter_name))
    conn.commit()
    new_id = cursor.lastrowid
    conn.close()

    return jsonify({
        "success": True,
        "message": "Complaint logged successfully",
        "id": new_id
    }), 201

@app.route('/api/complaints/<int:complaint_id>', methods=['PATCH', 'PUT'])
def update_complaint(complaint_id):
    """Update status and resolution notes of an existing complaint."""
    data = request.get_json()
    if not data:
        return jsonify({"success": False, "error": "Invalid request body"}), 400

    status = data.get('status')
    resolution_notes = data.get('resolution_notes')

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM complaints WHERE id = ?", (complaint_id,))
    complaint = cursor.fetchone()

    if not complaint:
        conn.close()
```

```
        return jsonify({"success": False, "error": "Complaint not found"}), 404

    new_status = status if status else complaint['status']
    new_notes = resolution_notes if resolution_notes is not None else complaint['resolution_notes']

    cursor.execute('''
        UPDATE complaints
        SET status = ?, resolution_notes = ?
        WHERE id = ?
    ''', (new_status, new_notes, complaint_id))
    conn.commit()
    conn.close()

    return jsonify({"success": True, "message": "Complaint updated successfully"}), 200

@app.route('/api/complaints/<int:complaint_id>', methods=['DELETE'])
def delete_complaint(complaint_id):
    """Delete a complaint record."""
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id FROM complaints WHERE id = ?", (complaint_id,))
    if not cursor.fetchone():
        conn.close()
        return jsonify({"success": False, "error": "Complaint not found"}), 404

    cursor.execute("DELETE FROM complaints WHERE id = ?", (complaint_id,))
    conn.commit()
    conn.close()

    return jsonify({"success": True, "message": "Complaint deleted successfully"}), 200

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

database.py — SQLite connection and table creation · 31 lines

```
import sqlite3

DATABASE_NAME = 'complaints.db'

def get_db_connection():
    conn = sqlite3.connect(DATABASE_NAME)
    conn.row_factory = sqlite3.Row  # Returns rows as dictionary-like objects
    return conn

def init_db():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS complaints (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            category TEXT NOT NULL,
            location TEXT NOT NULL,
            description TEXT NOT NULL,
            reporter_name TEXT NOT NULL,
            status TEXT DEFAULT 'Pending',
            resolution_notes TEXT DEFAULT '',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    ''')
    conn.commit()
    conn.close()

if __name__ == '__main__':
    init_db()
    print("Database initialized successfully.")
```

requirements.txt — Python dependency · 1 line

```
Flask==3.0.0
```

templates/index.html — Page structure · 144 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Campus Maintenance Portal</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <!-- Google Fonts for clean modern typography -->
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet">
</head>
<body>

    <header class="navbar">
        <div class="container nav-content">
            <h2>Campus Maintenance & Complaint Portal</h2>
            <span class="badge-team">Group 4: Kush | Kinshuk | Akshat</span>
        </div>
    </header>

    <div class="container main-content">
        <!-- Metrics Stats Bar -->
        <section class="stats-grid">
            <div class="card stat-card">
                <h3>Total Complaints</h3>
                <p id="stat-total">0</p>
            </div>
            <div class="card stat-card pending">
                <h3>Pending</h3>
                <p id="stat-pending">0</p>
            </div>
            <div class="card stat-card in-progress">
                <h3>In Progress</h3>
                <p id="stat-in-progress">0</p>
            </div>
            <div class="card stat-card resolved">
                <h3>Resolved</h3>
                <p id="stat-resolved">0</p>
            </div>
        </section>

        <!-- Main Layout: Form (Left) & Complaint Records (Right) -->
        <div class="dashboard-grid">

            <!-- Complaint Submission Form -->
            <div class="card form-card">
                <h3>Lodge a Complaint</h3>
                <p class="subtitle">Report broken facilities and campus issues</p>
                <form id="complaint-form">
                    <div class="form-group">
                        <label for="reporter_name">Your Name / Roll No.</label>
                        <input type="text" id="reporter_name" placeholder="e.g. Kush (2026UCD3136)" required>
                    </div>

                    <div class="form-group">
                        <label for="title">Issue Summary</label>
                        <input type="text" id="title" placeholder="e.g. AC leaking water in Lab 3" required>
                    </div>

                    <div class="form-group">
                        <label for="category">Category</label>
                        <select id="category" required>
```

```
                            <option value="">Select Category</option>
                            <option value="Electrical">Electrical</option>
                            <option value="Plumbing">Plumbing</option>
                            <option value="IT & Internet">IT & Internet</option>
                            <option value="Furniture/Carpentry">Furniture / Carpentry</option>
                            <option value="Cleanliness">Cleanliness</option>
                            <option value="Other">Other</option>
                        </select>
                    </div>

                    <div class="form-group">
                        <label for="location">Location / Room Number</label>
                        <input type="text" id="location" placeholder="e.g. Block B, 2nd Floor, Room 204"
required>
                    </div>

                    <div class="form-group">
                        <label for="description">Detailed Description</label>
                        <textarea id="description" rows="3" placeholder="Describe the severity and issue..."
required></textarea>
                    </div>

                    <button type="submit" class="btn btn-primary" id="submit-btn">Submit Ticket</button>
                </form>
            </div>

            <!-- Complaint Tracker & List -->
            <div class="card list-card">
                <div class="list-header">
                    <h3>Active Campus Tickets</h3>
                    <div class="filters">
                        <select id="filter-status" onchange="fetchComplaints()">
                            <option value="">All Statuses</option>
                            <option value="Pending">Pending</option>
                            <option value="In Progress">In Progress</option>
                            <option value="Resolved">Resolved</option>
                        </select>
                        <select id="filter-category" onchange="fetchComplaints()">
                            <option value="">All Categories</option>
                            <option value="Electrical">Electrical</option>
                            <option value="Plumbing">Plumbing</option>
                            <option value="IT & Internet">IT & Internet</option>
                            <option value="Furniture/Carpentry">Furniture/Carpentry</option>
                            <option value="Cleanliness">Cleanliness</option>
                            <option value="Other">Other</option>
                        </select>
                    </div>
                </div>

                <div id="complaints-list" class="complaints-container">
                    <!-- Dynamic complaints populated by app.js -->
                </div>
            </div>

        </div>
    </div>

    <!-- Edit Resolution Modal -->
    <div id="updateModal" class="modal">
        <div class="modal-content">
            <span class="close-modal" onclick="closeModal()">&times;</span>
            <h3>Update Maintenance Status</h3>
            <input type="hidden" id="modal-complaint-id">

            <div class="form-group">
                <label for="modal-status">Change Status</label>
                <select id="modal-status">
                    <option value="Pending">Pending</option>
                    <option value="In Progress">In Progress</option>
                    <option value="Resolved">Resolved</option>
                </select>
            </div>
```

```
132
133             <div class="form-group">
134                 <label for="modal-notes">Resolution Notes / Action Taken</label>
135                 <textarea id="modal-notes" rows="3" placeholder="e.g. Technician replaced capacitor on Nov
    3rd."></textarea>
136             </div>
137
138             <button class="btn btn-success" onclick="saveStatusUpdate()">Save Update</button>
139         </div>
140     </div>
141
142     <script src="{{ url_for('static', filename='js/app.js') }}"></script>
143 </body>
144 </html>
```

static/js/app.js — Frontend logic and API requests · 183 lines

```
1   document.addEventListener('DOMContentLoaded', () => {
2       fetchStats();
3       fetchComplaints();
4
5       // Attach form submission handler
6       const complaintForm = document.getElementById('complaint-form');
7       complaintForm.addEventListener('submit', handleCreateComplaint);
8   });
9
10  // GET: Fetch dashboard metrics
11  async function fetchStats() {
12      try {
13          const response = await fetch('/api/stats');
14          const data = await response.json();
15
16          document.getElementById('stat-total').innerText = data.total;
17          document.getElementById('stat-pending').innerText = data.pending;
18          document.getElementById('stat-in-progress').innerText = data.in_progress;
19          document.getElementById('stat-resolved').innerText = data.resolved;
20      } catch (error) {
21          console.error("Error fetching stats:", error);
22      }
23  }
24
25  // GET: Fetch and display complaints with optional filters
26  async function fetchComplaints() {
27      const status = document.getElementById('filter-status').value;
28      const category = document.getElementById('filter-category').value;
29
30      let url = `/api/complaints?`;
31      if (status) url += `status=${encodeURIComponent(status)}&`;
32      if (category) url += `category=${encodeURIComponent(category)}`;
33
34      try {
35          const response = await fetch(url);
36          const resData = await response.json();
37
38          const container = document.getElementById('complaints-list');
39          container.innerHTML = '';
40
41          if (!resData.data || resData.data.length === 0) {
42              container.innerHTML = `<p style="color: #64748b; text-align: center; padding: 20px;">No
    complaints found.</p>`;
43              return;
44          }
45
46          resData.data.forEach(item => {
47              const card = document.createElement('div');
48              card.className = 'ticket-card';
49
50              const badgeClass = item.status === 'Resolved'
51                  ? 'tag-resolved'
```

```
                : item.status === 'In Progress'
                    ? 'tag-in-progress'
                    : 'tag-pending';

            card.innerHTML = `
                <div class="ticket-top">
                    <h4>${escapeHtml(item.title)}</h4>
                    <span class="tag ${badgeClass}">${item.status}</span>
                </div>
                <div class="ticket-meta">
                    <span>📍 <strong>Location:</strong> ${escapeHtml(item.location)}</span>
                    <span>📁 <strong>Category:</strong> ${escapeHtml(item.category)}</span>
                    <span>👤 <strong>By:</strong> ${escapeHtml(item.reporter_name)}</span>
                </div>
                <p style="font-size: 0.9rem; color: #334155;">${escapeHtml(item.description)}</p>

                ${item.resolution_notes ? `
                    <div class="resolution-box">
                        <strong>Resolution Note:</strong> ${escapeHtml(item.resolution_notes)}
                    </div>
                ` : ''}

                <div class="ticket-actions">
                    <button class="btn btn-secondary" onclick="openUpdateModal(${item.id}, '${item.status}', \`${escapeHtml(item.resolution_notes || '')}\`)">Update Status</button>
                    <button class="btn btn-danger" onclick="deleteComplaint(${item.id})">Delete</button>
                </div>
            `;
            container.appendChild(card);
        });
    } catch (error) {
        console.error("Error loading complaints:", error);
    }
}

// POST: Add a new complaint
async function handleCreateComplaint(e) {
    e.preventDefault();

    const payload = {
        reporter_name: document.getElementById('reporter_name').value,
        title: document.getElementById('title').value,
        category: document.getElementById('category').value,
        location: document.getElementById('location').value,
        description: document.getElementById('description').value
    };

    try {
        const response = await fetch('/api/complaints', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });

        const data = await response.json();
        if (data.success) {
            document.getElementById('complaint-form').reset();
            fetchComplaints();
            fetchStats();
        } else {
            alert(data.error || "Failed to submit ticket");
        }
    } catch (err) {
        console.error("Error posting complaint:", err);
    }
}

// PATCH/PUT: Update Status & Resolution Notes Modal Handling
function openUpdateModal(id, currentStatus, currentNotes) {
    document.getElementById('modal-complaint-id').value = id;
    document.getElementById('modal-status').value = currentStatus;
    document.getElementById('modal-notes').value = currentNotes;
```

```
    document.getElementById('updateModal').style.display = 'flex';
}

function closeModal() {
    document.getElementById('updateModal').style.display = 'none';
}

async function saveStatusUpdate() {
    const id = document.getElementById('modal-complaint-id').value;
    const status = document.getElementById('modal-status').value;
    const resolution_notes = document.getElementById('modal-notes').value;

    try {
        const response = await fetch(`/api/complaints/${id}`, {
            method: 'PATCH',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ status, resolution_notes })
        });

        const data = await response.json();
        if (data.success) {
            closeModal();
            fetchComplaints();
            fetchStats();
        } else {
            alert(data.error || "Failed to update record");
        }
    } catch (err) {
        console.error("Error updating complaint:", err);
    }
}

// DELETE: Delete complaint
async function deleteComplaint(id) {
    if (!confirm("Are you sure you want to delete this complaint?")) return;

    try {
        const response = await fetch(`/api/complaints/${id}`, {
            method: 'DELETE'
        });
        const data = await response.json();
        if (data.success) {
            fetchComplaints();
            fetchStats();
        } else {
            alert(data.error || "Failed to delete record");
        }
    } catch (err) {
        console.error("Error deleting complaint:", err);
    }
}

// Security utility: escape HTML to avoid XSS
function escapeHtml(str) {
    if (!str) return '';
    return str.replace(/&/g, "&amp;")
              .replace(/</g, "&lt;")
              .replace(/>/g, "&gt;")
              .replace(/"/g, "&quot;")
              .replace(/'/g, "&#039;");
}
```

static/css/style.css — Styling and responsive layout · 268 lines

```
:root {
    --primary: #2563eb;
    --primary-hover: #1d4ed8;
    --bg-page: #f8fafc;
    --card-bg: #ffffff;
```

```
    --text-main: #0f172a;
    --text-muted: #64748b;
    --border-color: #e2e8f0;

    --pending-color: #f59e0b;
    --progress-color: #3b82f6;
    --resolved-color: #10b981;
}

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', sans-serif;
}

body {
    background-color: var(--bg-page);
    color: var(--text-main);
    line-height: 1.5;
}

.container {
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 1rem;
}

.navbar {
    background: #1e293b;
    color: white;
    padding: 1.2rem 0;
}

.nav-content {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.badge-team {
    font-size: 0.85rem;
    background: #334155;
    padding: 4px 10px;
    border-radius: 20px;
}

/* Stats section */
.stats-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 1rem;
    margin: 1.5rem 0;
}

.card {
    background: var(--card-bg);
    border-radius: 8px;
    padding: 1.2rem;
    border: 1px solid var(--border-color);
    box-shadow: 0 1px 3px rgba(0,0,0,0.05);
}

.stat-card h3 {
    font-size: 0.9rem;
    color: var(--text-muted);
}

.stat-card p {
    font-size: 1.8rem;
    font-weight: 700;
    margin-top: 5px;
```

```
}

.stat-card.pending p { color: var(--pending-color); }
.stat-card.in-progress p { color: var(--progress-color); }
.stat-card.resolved p { color: var(--resolved-color); }

/* Main Dashboard layout */
.dashboard-grid {
    display: grid;
    grid-template-columns: 380px 1fr;
    gap: 1.5rem;
    align-items: start;
    margin-bottom: 2rem;
}

@media (max-width: 900px) {
    .dashboard-grid {
        grid-template-columns: 1fr;
    }
}

.subtitle {
    font-size: 0.85rem;
    color: var(--text-muted);
    margin-bottom: 1rem;
}

.form-group {
    margin-bottom: 1rem;
}

.form-group label {
    display: block;
    font-size: 0.85rem;
    font-weight: 600;
    margin-bottom: 4px;
}

.form-group input, .form-group select, .form-group textarea {
    width: 100%;
    padding: 8px 10px;
    border: 1px solid var(--border-color);
    border-radius: 6px;
    font-size: 0.9rem;
}

.btn {
    padding: 10px 16px;
    border: none;
    border-radius: 6px;
    font-weight: 600;
    cursor: pointer;
    transition: 0.2s ease;
}

.btn-primary {
    background-color: var(--primary);
    color: white;
    width: 100%;
}
.btn-primary:hover { background-color: var(--primary-hover); }

.btn-secondary { background: #e2e8f0; color: #334155; }
.btn-secondary:hover { background: #cbd5e1; }

.btn-danger { background: #ef4444; color: white; }
.btn-danger:hover { background: #dc2626; }

.btn-success { background: var(--resolved-color); color: white; width: 100%; }

/* List Header */
.list-header {
```

```
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: 10px;
    margin-bottom: 1rem;
    border-bottom: 1px solid var(--border-color);
    padding-bottom: 0.8rem;
}

.filters select {
    padding: 6px 8px;
    border-radius: 6px;
    border: 1px solid var(--border-color);
}

/* Complaints Cards */
.complaints-container {
    display: flex;
    flex-direction: column;
    gap: 1rem;
    max-height: 650px;
    overflow-y: auto;
}

.ticket-card {
    border: 1px solid var(--border-color);
    border-radius: 8px;
    padding: 1rem;
    background: #ffffff;
    transition: box-shadow 0.2s;
}

.ticket-card:hover {
    box-shadow: 0 4px 6px rgba(0,0,0,0.07);
}

.ticket-top {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 0.5rem;
}

.ticket-top h4 {
    font-size: 1.05rem;
}

.tag {
    font-size: 0.75rem;
    padding: 3px 8px;
    border-radius: 12px;
    font-weight: 600;
}

.tag-pending { background: #fef3c7; color: #b45309; }
.tag-in-progress { background: #dbeafe; color: #1d4ed8; }
.tag-resolved { background: #d1fae5; color: #065f46; }

.ticket-meta {
    font-size: 0.8rem;
    color: var(--text-muted);
    margin-bottom: 0.5rem;
}

.ticket-meta span {
    margin-right: 12px;
}

.resolution-box {
    margin-top: 0.6rem;
    background: #f1f5f9;
```

```
    padding: 6px 10px;
    border-left: 3px solid var(--resolved-color);
    border-radius: 4px;
    font-size: 0.85rem;
}

.ticket-actions {
    margin-top: 0.8rem;
    display: flex;
    gap: 8px;
}

.ticket-actions .btn {
    padding: 4px 10px;
    font-size: 0.8rem;
}

/* Modal styles */
.modal {
    display: none;
    position: fixed;
    z-index: 1000;
    left: 0;
    top: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    align-items: center;
    justify-content: center;
}

.modal-content {
    background-color: white;
    padding: 1.5rem;
    border-radius: 8px;
    width: 90%;
    max-width: 450px;
    position: relative;
}

.close-modal {
    position: absolute;
    right: 15px;
    top: 10px;
    font-size: 1.5rem;
    cursor: pointer;
}
```